

KOREVALUE · TECHNICAL GUIDE · 2026

AI Infrastructure Optimization

A technical playbook for maximizing GPU utilization, reducing AI workload costs, and scaling intelligence efficiently

Utilize · Automate · Optimize · Scale

For ML Engineers	For Platform Teams	For Infrastructure Leads
------------------	--------------------	--------------------------

Confidential · KoreValue · 2026

NAVIGATION

Table of Contents

Table of Contents	2
Why AI Infrastructure Optimization Is Urgent	3
The Numbers Behind the Urgency	3
GPU Utilization Optimization	4
Understanding GPU Utilization	4
Optimization Techniques: Training Workloads	4
Optimization Techniques: Inference Workloads	4
Intelligent Workload Scheduling	6
The Scheduling Problem	6
Scheduling Strategies	6
Scheduling for Cost at Scale	6
Model Serving Optimization	7
Serving Architecture Patterns	7
Serving Infrastructure Optimization	7
Cost Monitoring for AI Infrastructure	8
The Core Metrics	8
Monitoring Architecture	8
Optimization Feedback Loop	8
The Optimization Roadmap	9
Phase 1 — Foundation (Weeks 1–4)	9
Phase 2 — Quick Wins (Weeks 5–8)	9
Phase 3 — Systematic Optimization (Weeks 9–16)	9
Phase 4 — Advanced Efficiency (Month 5+)	9
Conclusion	10
Contact Us	10

SECTION 01

Why AI Infrastructure Optimization Is Urgent

GPU compute is the most expensive, most constrained, and most wasted resource in modern AI organizations. Optimizing it is not a cost exercise — it is a strategic imperative.

The economics of AI infrastructure have reached an inflection point. As model sizes grow, inference demands scale, and GPU supply remains constrained, organizations face a dual pressure: costs are rising while availability is tightening. The teams that get more from every GPU-hour will outpace those that simply provision more.

AI infrastructure optimization is the discipline of extracting maximum value from every compute resource — through better scheduling, smarter architecture, automated rightsizing, and continuous performance monitoring. It is the difference between an AI initiative that scales and one that bankrupts.

The Numbers Behind the Urgency

43% Average GPU idle rate in unmanaged AI infrastructure	\$2.1M Average annual GPU waste per 100-GPU enterprise cluster	67% Of AI projects exceed infrastructure cost projections	3.4x Performance improvement achievable through systematic optimization
--	--	---	---

The gap between optimized and unoptimized AI infrastructure is not marginal. Organizations that systematically optimize GPU utilization, workload scheduling, and model serving architecture operate at a fundamentally different cost basis — enabling them to run more experiments, serve more users, and scale faster on the same budget.

SECTION 02

GPU Utilization Optimization

The GPU is the engine of modern AI. Every percentage point of utilization improvement translates directly to dollars saved and capability unlocked.

Understanding GPU Utilization

GPU utilization is not binary — it exists across a spectrum. A GPU can be powered on (paying), but idle (no active kernels). It can have active kernels, but with poor memory bandwidth utilization. It can be fully utilized at the compute level but bottlenecked by CPU preprocessing. Understanding where utilization is lost is the first step to recovering it.

Idle GPU	GPU powered on with no active workloads. Pure waste — full cost, zero value. Most common cause: poor job scheduling and lack of scale-to-zero.
Low Utilization	GPU running but at under 30% compute utilization. Common with small batch sizes, inefficient data loading, or poorly parallelized code.
Memory Bottleneck	Compute units waiting for memory operations. Caused by large model sizes, inefficient memory access patterns, or insufficient VRAM.
CPU Bottleneck	GPU waiting for CPU to complete preprocessing or postprocessing. Data pipelines that cannot keep the GPU fed.
Fully Optimized	80%+ sustained GPU utilization with balanced memory bandwidth. The target state for every production workload.

Optimization Techniques: Training Workloads

Training jobs are the highest-cost and most optimization-amenable workloads in AI infrastructure. Small improvements compound over long training runs.

- ▶ Mixed precision training (FP16/BF16) — reduces memory footprint by 50%, enables larger batch sizes, typically 1.5–2x throughput improvement
- ▶ Gradient accumulation — simulate large batch sizes on smaller GPU memory, enabling more efficient GPU utilization
- ▶ Data parallelism and model parallelism — distribute workloads across multiple GPUs to eliminate single-GPU bottlenecks
- ▶ Efficient data loading — use asynchronous data pipelines with prefetching to ensure GPUs are never waiting on data
- ▶ Checkpoint optimization — implement frequent checkpointing to minimize waste from job failures on long training runs
- ▶ Spot and preemptible GPUs — use interruptible instances for fault-tolerant training jobs at 60–80% cost reduction

Optimization Techniques: Inference Workloads

Inference is where AI delivers value — and where cost can silently spiral. Optimization at the inference layer has compounding returns as request volume grows.

- ▶ Model quantization — INT8 and INT4 quantization reduces model size and memory requirements with minimal accuracy loss
- ▶ Batched inference — process multiple requests simultaneously to maximize GPU utilization per request served
- ▶ KV cache optimization — reduce redundant computation in transformer-based models through intelligent key-value caching

- ▶ Model distillation — train smaller, faster models that approximate larger model outputs for latency-sensitive applications
- ▶ Dynamic batching — automatically group incoming requests to optimize GPU utilization without increasing latency
- ▶ Scale-to-zero — terminate inference endpoints with no traffic; cold start on demand for cost efficiency in variable-traffic scenarios

SECTION 03

Intelligent Workload Scheduling

The right workload on the right hardware at the right time. Scheduling is the highest-leverage optimization lever available — it costs nothing but intelligence.

The Scheduling Problem

Most AI infrastructure operates with naive scheduling: workloads are submitted and run immediately on whatever resource is available. This approach ignores workload characteristics, resource availability patterns, cost differentials, and priority hierarchies — resulting in suboptimal utilization, unnecessary costs, and priority inversions where low-value jobs consume expensive resources.

Intelligent scheduling treats workload placement as an optimization problem. It matches workload requirements to resource capabilities, timing to cost windows, and priority to business value — automatically, at scale, continuously.

Scheduling Strategies

- ▶ Priority-based queuing — define workload tiers (production, development, experimentation) and schedule accordingly; prevent low-priority jobs from consuming premium resources
- ▶ Spot-aware scheduling — route fault-tolerant training jobs to spot instances automatically; fall back to on-demand when spot availability drops
- ▶ Time-shifting — schedule non-urgent batch jobs to off-peak hours when GPU spot prices are lowest
- ▶ Resource matching — match GPU type to workload requirements; avoid A100 costs for workloads that run equally well on T4s
- ▶ Gang scheduling — ensure multi-GPU jobs receive all required resources simultaneously, eliminating partial allocation waste
- ▶ Preemption policies — allow high-priority production jobs to preempt lower-priority workloads with checkpoint preservation

Scheduling for Cost at Scale

Scheduling Action	Cost Impact
Route training to spot GPUs	60–80% reduction in training compute costs
Time-shift batch jobs to off-peak	20–40% reduction in on-demand GPU costs
Match GPU type to workload need	30–50% cost reduction vs. always using top-tier GPUs
Implement scale-to-zero for idle endpoints	100% savings on idle inference infrastructure
Enable preemption for dev workloads	25–35% improvement in production GPU utilization

SECTION 04

Model Serving Optimization

The way a model is served determines its cost, latency, and scalability. Serving architecture is infrastructure strategy.

Serving Architecture Patterns

There is no single correct way to serve an AI model. The right architecture depends on latency requirements, traffic patterns, model size, and cost targets. Most organizations use a single serving pattern for all workloads — leaving significant optimization opportunity on the table.

Dedicated Endpoint	Single model on dedicated GPU. Best for: high-traffic, latency-sensitive production models. Cost: highest. Latency: lowest.
Shared Multi-Model	Multiple models share GPU resources. Best for: low-traffic models with burst patterns. Cost: low. Requires: careful resource management.
Serverless Inference	Scale-to-zero with on-demand startup. Best for: infrequent or unpredictable traffic. Cost: near-zero at idle. Latency: cold start overhead.
Streaming Inference	Response streaming for long-generation tasks. Best for: LLM applications. Improves perceived latency without reducing actual compute time.
Cached Inference	Cache frequent or identical requests. Best for: high-repetition query patterns. Can eliminate GPU compute entirely for repeated queries.

Serving Infrastructure Optimization

- ▶ Model versioning with shadow traffic — test new model versions on a fraction of live traffic before full rollout, with automatic rollback on performance degradation
- ▶ Horizontal autoscaling — scale replica count based on request queue depth and GPU utilization, not fixed capacity
- ▶ Request prioritization — implement priority queues at the serving layer to ensure SLA compliance for premium-tier requests
- ▶ Inference caching layer — deploy semantic caching to serve repeated or similar queries from cache, reducing GPU load by up to 40% for high-repetition use cases
- ▶ Multi-region serving — route requests to lowest-latency, lowest-cost region based on real-time capacity and pricing
- ▶ Continuous benchmarking — measure serving performance against cost and latency SLAs on every deployment, with automated rollback on regression

SECTION 05

Cost Monitoring for AI Infrastructure

Optimization without measurement is guesswork. The metrics that matter — and how to track them.

The Core Metrics

GPU Utilization Rate	Percentage of GPU compute actively used. Target: 80%+ for production workloads. Below 30% indicates significant rightsizing opportunity.
Cost Per Training Run	Total GPU cost attributed to each model training job. Enables comparison across runs and teams.
Cost Per Inference Request	Infrastructure cost per served request. The fundamental unit economics metric for AI products.
GPU Idle Rate	Percentage of GPU-hours with no active workloads. Directly quantifies waste in dollar terms.
Utilization Efficiency Score	Composite metric combining GPU compute, memory bandwidth, and throughput utilization. Holistic view of hardware efficiency.
Time-to-GPU	Latency from job submission to GPU allocation. High values indicate scheduling inefficiency or capacity constraints.

Monitoring Architecture

Effective AI infrastructure monitoring requires telemetry at multiple layers — hardware, workload, and cost — integrated into a unified view.

- ▶ Hardware telemetry: GPU utilization, memory usage, temperature, and power draw — sampled at sub-minute intervals
- ▶ Workload telemetry: job status, duration, resource consumption, and cost attribution per job and per team
- ▶ Cost telemetry: real-time dollar spend by resource, workload, team, and environment — with anomaly detection on all dimensions
- ▶ Performance telemetry: inference latency, throughput, error rates, and SLA compliance — enabling cost-performance tradeoff analysis
- ▶ Capacity telemetry: cluster utilization trends, queue depth, and resource contention — informing scaling and scheduling decisions

Optimization Feedback Loop

Monitoring only creates value when it drives action. The optimization feedback loop connects measurement to improvement continuously:

- ▶ Measure: collect real-time metrics across all infrastructure layers
- ▶ Analyze: identify utilization gaps, cost anomalies, and optimization opportunities automatically
- ▶ Recommend: surface ranked optimization actions with estimated impact per recommendation
- ▶ Act: implement approved recommendations — automatically for low-risk actions, with approval workflow for architectural changes
- ▶ Verify: confirm optimization impact against baseline metrics and close the loop

SECTION 06

The Optimization Roadmap

Systematic optimization compounds. Here is the sequence that maximizes return and minimizes risk.

Phase 1 — Foundation (Weeks 1–4)

Establish visibility before touching anything. No optimization action should precede a clear baseline understanding of current infrastructure state.

- ▶ Deploy cost and utilization monitoring across all GPU workloads
- ▶ Establish baseline metrics: current GPU utilization rate, idle rate, cost-per-job
- ▶ Identify top 10 highest-cost workloads and current resource consumption
- ▶ Audit current scheduling policies and identify obvious inefficiencies

Phase 2 — Quick Wins (Weeks 5–8)

Target the highest-impact, lowest-complexity opportunities first. These generate momentum and fund more complex optimization work.

- ▶ Implement scale-to-zero for all idle inference endpoints — immediate 100% savings on idle time
- ▶ Right-size oversized training instances based on actual utilization data
- ▶ Route eligible training jobs to spot instances — 60–80% compute cost reduction
- ▶ Set automated timeout policies on training jobs to eliminate runaway costs

Phase 3 — Systematic Optimization (Weeks 9–16)

With quick wins captured, move to structural improvements that deliver sustained, compounding savings.

- ▶ Implement mixed precision training across all eligible training workloads
- ▶ Deploy intelligent scheduling with priority tiers and resource matching
- ▶ Introduce batched and dynamic inference serving for high-traffic models
- ▶ Establish automated recommendations engine with weekly optimization review cycle

Phase 4 — Advanced Efficiency (Month 5+)

The frontier of AI infrastructure optimization — techniques that require maturity to implement but deliver disproportionate returns.

- ▶ Model quantization and distillation for latency-sensitive inference workloads
- ▶ Multi-region cost-aware inference routing
- ▶ Semantic caching layer for high-repetition query patterns
- ▶ Continuous benchmarking with automated regression detection and rollback

SECTION 07

Conclusion

AI infrastructure optimization is not a one-time project. It is a continuous practice — and the organizations that build it as a capability will operate at a permanently lower cost basis.

The techniques in this guide are not theoretical. They are battle-tested across GPU clusters, cloud providers, and AI workload types. Organizations that implement them systematically can expect to reduce AI infrastructure costs by 30–60% within 90 days — while improving performance and scaling capacity simultaneously.

The key is sequencing. Visibility before optimization. Quick wins before architectural changes. Automation before manual governance. KoreValue provides the platform to execute every phase of this roadmap — from baseline monitoring through fully automated, AI-driven continuous optimization.

The GPU is the scarcest and most expensive resource in the AI era. The organizations that use it best will build the most. Start optimizing today.

KoreValue · AI Infrastructure Optimization · v1.0 · Confidential

SECTION 08

Contact Us

For inquiries regarding our AI FinOps roadmap, partnership opportunities, or technical architecture, please reach out to our core team

Admin@CtrlS.co